PART 4

Prompt Engineering

Six chapters on what to put in the request: the principles that apply to every current model, the system prompt and the role, XML structure, output contracts you can rely on, the evaluation suite that turns opinion into measurement, and the guardrails that catch what is left.

IN THIS PART

- 20. Principles of Prompting
- 21. System Prompts and Role Design
- 22. Structured Prompting with XML
- 23. Structured Output and JSON Contracts
- 24. Prompt Evaluation and Regression Suites
- 25. Guardrails, Validation and Prompt Debugging

CHAPTER 20

Principles of Prompting

In this chapter

Introduction: The Tidy Prompt That Cost $144,660 a Month . . . 4
20.1 What Prompt Engineering Is For, and When It Is the Wrong Tool . . . 5
20.2 The Golden Rule . . . 6
20.3 Be Clear and Direct . . . 6
20.4 Give the Reason, Not Just the Rule . . . 7
20.5 Say What to Do, Not What Not to Do . . . 7
20.6 Examples: Relevant, Diverse, Structured . . . 8
20.7 A Prompt Has Parts . . . 8
20.8 Order Is Not Cosmetic . . . 9
20.9 Grounding in Quotes . . . 10
20.10 Roles . . . 10
20.11 Verbosity, Format and the Models' Current Defaults . . . 11
20.12 Prompting Is Not Configuration . . . 11
20.13 Model Self-Knowledge . . . 12
20.14 What Changed, and What to Delete . . . 12
20.15 Visual Analogies . . . 13
20.16 Architecture: The Prompt as a Build Artefact . . . 14
20.17 Examples: Four Rewrites . . . 15
20.18 Code Walkthrough: A Prompt That Knows Its Own Shape . . . 16
20.19 Mini Project: The Prompt Compiler and Linter . . . 17
20.20 Panel A — Where a Prompt's Tokens Actually Are . . . 18
20.21 Panel B — The Same Content, Five Orderings . . . 18
20.22 Panel C — What the Ordering Costs . . . 19
20.23 Panel D — The Linter on a Draft and Its Rewrite . . . 20
20.24 Panel E — The Linter on the Five Arrangements . . . 21
20.25 Panel F — The One Arrangement That Is Both Clean and Cheapest . . . 21
20.26 Evaluation and Deployment . . . 22
20.27 Hands-on Labs . . . 22
20.28 Lab Solutions . . . 22

20.29 Best Practices 23
20.30 Common Mistakes 24
20.31 Performance Considerations 25
20.32 Security Considerations 26
20.33 Real Production Examples 26
20.34 Certification Notes 27
20.35 Twenty Practice Questions 28
20.36 Answers and Explanations 28
20.37 Ten Practical Exercises 29
20.38 Five Challenge Labs 30
20.39 Ten Interview Questions 30
20.40 Summary 30
20.41 Glossary 32
20.42 Further Reading 33

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Say when prompt engineering is the right tool and when a different lever — a model, a parameter, a retrieval step — is cheaper.
2. Apply the **golden rule** of clear prompting and the four principles under it: be specific, give the reason, say what to do rather than what not to do, and show examples.
3. Decompose a prompt into **parts** with a kind and a stability, rather than treating it as a string.
4. Order a prompt correctly, and explain why the documented **quality** rule and the **caching** rule are the same rule.
5. Use examples effectively — relevant, diverse, structured — and know the documented count.
6. Steer output **format and verbosity** without negation, and ground long-document work in quotes.
7. Distinguish prompting from **configuration**: what belongs in the prompt and what belongs in `effort`, `thinking` and `max_tokens`.
8. Rewrite a prompt written for an earlier generation, removing prefill, anti-laziness language and over-emphasis.

NOTE

Prerequisites

Chapter 6 for tokens as the unit of everything, and for `count_tokens`. **Chapter 7** for the escalation ladder that puts prompting before fine-tuning. **Chapter 8** for hallucination and for the discipline of measuring rather than believing. **Chapter 10** for the capability triage — information, exact execution, external state, checkability — in which prompting is the fall-through. **Chapter 15** for evaluation sets, because a prompt you have not evaluated is a prompt you have an opinion about. **Chapter 16** for the request body and the top-level `system` field. **Chapter 18** for `effort` and thinking, which are not prompting. **Chapter 19** for the ITPM bucket, the spend cap and the cacheable prefix, all three of which this chapter's arithmetic lands on.

Part 3 asked what the API will accept. Part 4 asks what to put in it. This chapter is the foundation: the general principles that apply to every current model. System prompts and roles get Chapter 21, XML structuring Chapter 22, structured output and JSON contracts Chapter 23, evaluation and regression suites Chapter 24, and guardrails and debugging Chapter 25.

Introduction: The Tidy Prompt That Cost $144,660 a Month

A team writes a support-triage prompt. It reads well. Instructions at the top, so a human opening the file sees the task first. The output format next. Three worked examples. Then the ticket. Then, at the bottom, the twenty-thousand-token policy handbook the model needs to do the job — at the bottom, because that is where reference material goes in a document.

Every word of it is good. The instructions are specific, the examples are diverse, the output format is named. It is, as a piece of writing, better than most prompts in production anywhere.

It costs **$203,349 a month**, and the identical content in a different order costs **$58,689**.

Nothing is added and nothing is removed. Section 20.23 measures the whole comparison: same 21,487 tokens, same instructions, same examples, same handbook, same ticket. Moving the handbook from the bottom to the top takes the uncached tokens per request from **21,004 to 63** — a factor of **333.4** — the input-tokens-per-minute draw from 21,052 to 2,205, and the required usage tier from **Custom to Start** on rate limits alone. The saving is **$144,660 a month, or 71.1 per cent**, for a change that a reviewer would call cosmetic.

The reason is Chapter 19's, and it is mechanical. Prompt caching is **prefix-only**. The cacheable region of a prompt ends at the first thing that changes between requests. Put the ticket above the handbook and the handbook is *stranded*: twenty thousand tokens that are byte-identical on every single request, and that you pay for, and that draw your ITPM bucket, every single time, forever. The prompt is not badly written. It is badly **ordered**, and order is not a matter of taste.

There is a worse version, and it is the more common one. A template that interpolates the ticket into the instructions —

```
system = INSTRUCTIONS.format(ticket=ticket)   # tidy; catastrophic
```

— makes the *first* part of the prompt variable, so the cacheable prefix is **zero**. Section 20.23 prices it at **$206,730 a month**, the worst of the five arrangements tested, and section 20.25's linter flags it with two errors before it is ever sent.

And here is the part that makes this a chapter about principles rather than a chapter about caching. The order that is cheapest is **the order Anthropic already documents for quality reasons**: put longform data at the top, put the query at the bottom. The documentation reports that queries at the end improved response quality **by up to 30 per cent** in testing on complex multi-document inputs. The quality rule and the money rule are the same rule, arrived at independently, and a team that follows either one gets the other for nothing.

That is the shape of this chapter. Prompting is usually taught as a collection of tips. It is better understood as a small number of principles with measurable consequences — and the ones that can be measured offline, on your own machine, before a single token is sent, are measured here.

20.1 What Prompt Engineering Is For, and When It Is the Wrong Tool

Chapter 7 built an escalation ladder and put fine-tuning at the bottom. Prompting sits near the top of it, and the documentation is careful to say that it is not the answer to everything: *"Not every success criteria or failing eval is best solved by prompt engineering. For example, you can sometimes improve latency and cost more easily by selecting a different model."*

Chapter 10's triage is the sharper version of the same idea. Before rewriting a prompt, classify the failure:

The model lacks…	The fix is…	Chapter
Information it was never given	Retrieval or a document in context	53
Exact, repeatable execution	A tool, not a paragraph	Part 6

The model lacks…	The fix is…	Chapter
Access to external state	A tool or an MCP server	Parts 6–7
A way to be checked	A verifier, a schema, a test	23, 24
Nothing above — it simply misunderstood	**Prompting**	This chapter

Prompting is the fall-through, and that is not a demotion. Most failures in production systems *are* misunderstandings: the model did something reasonable that was not what you meant. But reaching for a prompt rewrite when the model is missing a fact, or is being asked to do arithmetic it cannot do reliably, is how teams accumulate two thousand words of instructions that fix nothing.

The documentation also states what must exist before you begin: a clear definition of success criteria, a way to test against them empirically, and a first draft to improve. Chapter 15 measured what happens when the second of those is skimped — a fifty-example evaluation set cost a modelled workload **$23,179 a month** in over-buying against a two-thousand-example one. Chapter 24 builds the regression suite properly. Until then, hold on to the principle: **a prompt change you cannot measure is a prompt change you cannot defend.**

20.2 The Golden Rule

The documentation gives one test, and it is worth more than any list of techniques:

> KEY TAKEAWAYS
>
> The golden rule of clear prompting
>
> **Show your prompt to a colleague with minimal context on the task and ask them to follow it. If they'd be confused, Claude will be too.**

The accompanying model is equally useful: think of Claude as *a brilliant but new employee who lacks context on your norms and workflows*. Brilliant, so you do not need to explain the domain from first principles. New, so you cannot assume they know which of your four billing systems "the ledger" refers to, that tickets from Enterprise accounts route differently, or that "handle it" means refund rather than escalate.

The golden rule is also the only one of this chapter's principles that a linter cannot check. Section 20.24's `AMBIGUOUS_OPENING` rule catches the crudest violation — a prompt that opens with an unbound pronoun — and that is genuinely all a machine can do. The rest is a colleague and five minutes.

20.3 Be Clear and Direct

Claude responds well to clear, explicit instructions, and being specific about the desired output improves results. The documented corollary is the one people find counter-intuitive: **if you want "above and beyond" behaviour, ask for it.** Do not expect it to be inferred from a vague prompt.

The documentation's own example:

	Prompt
Less effective	`Create an analytics dashboard`
More effective	`Create an analytics dashboard. Include as many relevant features and interactions as possible. Go beyond the basics to create a fully-featured implementation.`

Two specific practices sit under the principle.

Be specific about the desired output format and constraints. Not "summarise this" but "summarise this in three sentences of flowing prose, naming the account tier in the first sentence". Section 20.24's `NO_OUTPUT_SPEC` rule fires when a prompt names no format anywhere, which is the single most common finding on first drafts.

Provide instructions as sequential steps — a numbered list or bullet points — when the order or completeness of the steps matters. This is worth emphasising because the same documentation elsewhere warns against excessive markdown in *output*. The two are not in tension: numbered steps in your **instructions** are recommended; bulleted fragments in Claude's **response** are usually not what you want. Prompt formatting and response formatting are different decisions.

20.4 Give the Reason, Not Just the Rule

Providing context or motivation behind an instruction — explaining *why* the behaviour matters — helps Claude understand the goal and deliver a more targeted response. The documentation's example is the clearest statement of the principle in the whole guide:

	Prompt
Less effective	`NEVER use ellipses`
More effective	`Your response will be read aloud by a text-to-speech engine, so never use ellipses since the text-to-speech engine will not know how to pronounce them.`

The second version is longer, and it is better for a reason worth understanding. A bare rule covers exactly the case it names. A rule with a reason **generalises**: the model that knows the output is going to a speech engine will also avoid tables, emoji, "e.g.", and a bulleted list of URLs — none of which you thought to prohibit. The documentation puts it plainly: *Claude is smart enough to generalize from the explanation.*

This is also why the linter in section 20.24 treats a negation *with* a reason as clean and a negation *without* one as a warning. The negation is not the problem. The missing reason is.

20.5 Say What to Do, Not What Not to Do

The first and most effective way to steer output formatting, per the documentation:

	Prompt
Instead of	`Do not use markdown in your response`
Try	`Your response should be composed of smoothly flowing prose paragraphs.`

A prohibition defines a boundary and leaves everything inside it. A positive instruction defines a target. There is a second reason to prefer the positive form that Chapter 4 explains mechanically: attention operates over tokens that are present. "Do not use markdown" puts *markdown* in the context. It is a weak effect and it is not the main argument, but the direction is not helpful to you.

Two further documented techniques stack on top:

XML format indicators. *"Write the prose sections of your response in* `<smoothly_flowing_prose_paragraphs>` *tags."* Naming the container names the target. Chapter 22 develops this properly.

Match your prompt style to the desired output style. The formatting of your prompt influences the formatting of the response — so removing markdown from a prompt can reduce the volume of markdown in the output. This is why section 20.24 carries a `STYLE_MISMATCH` rule: a prompt that asks for flowing prose while itself being a bulleted list is working against itself.

20.6 Examples: Relevant, Diverse, Structured

Examples — few-shot or multishot prompting — are described in the documentation as one of the most reliable ways to steer output format, tone and structure. Three properties are named:

- **Relevant.** They mirror your actual use case. An example about invoices does not teach a model about refunds.
- **Diverse.** They cover edge cases and vary enough that Claude does not pick up an unintended pattern. If all three of your examples happen to end in `escalate`, you have taught escalation.
- **Structured.** Wrapped in `<example>` tags, nested inside `<examples>`, so Claude can tell them from instructions.

The documented count is **three to five**. Section 20.24 enforces the band in both directions, and the upper bound has a cost attached that the documentation does not mention: examples live in the stable prefix, so a sixth example is 150 tokens on every request forever. It will usually still be cached, and it is still not free.

Two refinements worth carrying:

You can ask Claude to critique your examples for relevance and diversity, or to generate more from your initial set. This is the cheapest possible use of the model in your development loop.

Examples work with thinking. Using `<thinking>` tags inside a few-shot example shows Claude the reasoning pattern you want, and it generalises that style to its own thinking blocks. Chapter 18 explained what those blocks are; this is how you shape them without configuring anything.

20.7 A Prompt Has Parts

Here is the reframing that the rest of the chapter depends on. A prompt is not a string. It is an **ordered list of parts**, and each part has two properties that decide everything:

Property	Values	Decides
Kind	instructions, output_spec, examples, documents, context, query	Which XML tag it gets, and where it belongs
Stability	stable or variable across requests	Whether it can live in the cacheable prefix

The kinds are not invented for this book; they are the tags the documentation recommends — `<instructions>`, `<context>`, `<examples>`, `<input>`, `<documents>`. What this chapter adds is the second column, and it is the one that is usually implicit and always consequential.

Section 20.21 measures where the tokens in a realistic prompt actually are:

Part	Tokens	Stability
documents (the policy handbook)	20,941	stable
examples	232	stable
instructions	183	stable
output_spec	68	stable
query (the ticket)	**63**	**variable**
total	**21,487**	

The part you write by hand is 483 tokens — 2.2 per cent of the prompt. The part that changes is 63 tokens — 0.29 per cent. Ninety-seven and a half per cent of every request is a document that is identical every time. Any reasoning about prompt cost that starts from the instructions is reasoning about the wrong two per cent.

20.8 Order Is Not Cosmetic

The documentation's long-context guidance, for inputs of roughly 20,000 tokens and up:

- **Put longform data at the top.** Place long documents and inputs near the top of your prompt, above your query, instructions and examples. *"Queries at the end can improve response quality by up to 30 percent in tests, especially with complex, multidocument inputs."*
- **Structure documents with XML tags** — each in a `<document>` with `<document_content>` and `<source>` subtags.
- **Ground responses in quotes** — ask Claude to quote the relevant parts first.

Now put that next to Chapter 11 and Chapter 19. Prompt caching is prefix-only. The cacheable region ends at the first part that changes. Therefore:

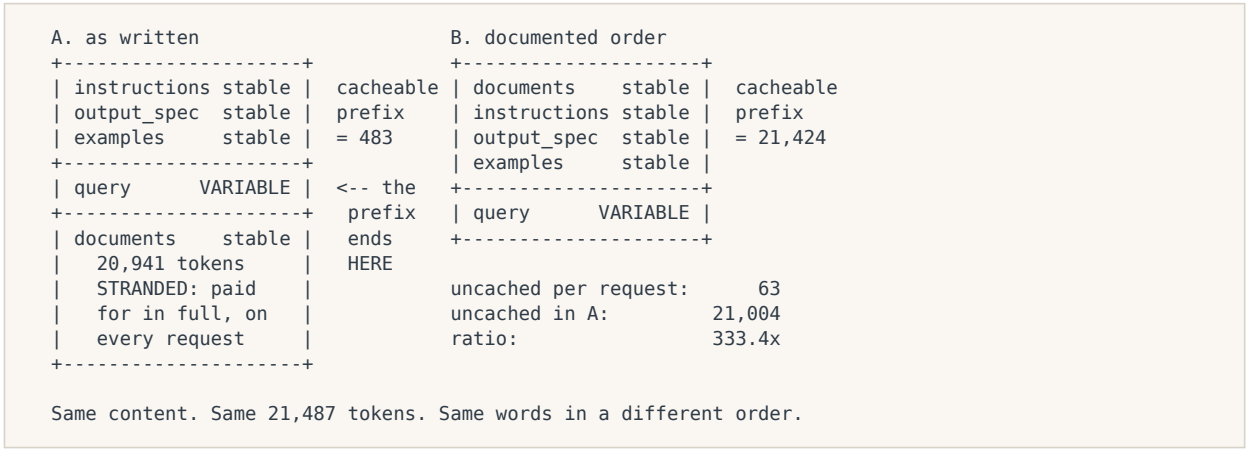


Figure 20.1 - Why the quality rule and the cache rule agree

The stranded region is the concept to carry. **Stable content placed after the first variable part is the most expensive kind of content there is**: identical on every request, and uncacheable, because the prefix already ended. Section 20.23 prices this one at $144,660 a month.

KEY TAKEAWAYS

A running thread for Part 4

Prompt order is an architectural decision, not an editorial one. Stable first, variable last — and the documented quality ordering and the caching ordering are the same ordering.

20.9 Grounding in Quotes

For long-document tasks the documentation recommends asking Claude to quote the relevant parts of the documents *before* carrying out the task, placing them in `<quotes>` tags. Its stated purpose is to help Claude focus on the relevant content and ignore the rest.

Three reasons to adopt it as a default rather than a tactic.

It is a focusing device. Chapter 5 established that attention has a fixed budget spread across the context; quoting first spends an explicit early step narrowing it.

It is a debugging device. When the answer is wrong, the quotes tell you whether the model failed to *find* the right passage or failed to *reason* from it. Those are different bugs with different fixes — retrieval versus prompting — and without the quotes you cannot tell them apart.

It is a grounding device, with a caveat. Chapter 8 measured exactly how far this goes: a lexical grounding check passes the most dangerous class of fabrication, because a quote can be real and the conclusion drawn from it still invented. Quoting improves the odds and does not close the hole. Chapter 8's citations feature is the stronger mechanism where it applies.

Section 20.25's `NO_QUOTE_GROUNDING` rule fires on any prompt with 20,000+ document tokens that never asks for quotes.

20.10 Roles

Setting a role in the system prompt focuses Claude's behaviour and tone, and the documentation notes that *even a single sentence makes a difference*:

```
{"model": "claude-opus-5", "max_tokens": 1024,
 "system": "You are a helpful coding assistant specializing in Python.",
 "messages": [{"role": "user", "content": "How do I sort dicts by key?"}]}
```

Two things to note now and develop in Chapter 21. First, `system` is a **top-level field**, not a message — Chapter 16's running thread, and the reason a role is architecture rather than conversation. Second, a role is cheap in tokens and lives at the very front of the stable prefix, which makes it the highest-leverage sentence in the whole prompt: it is read first, cached always, and costs a few dozen tokens once.

Chapter 21 covers what a role should and should not contain, operator authority, and why "you are a world-class expert" is weaker than a sentence describing the actual job.

20.11 Verbosity, Format and the Models' Current Defaults

The current generation communicates differently from its predecessors, and prompts tuned for older models now mis-steer. The documented differences:

- **More direct and grounded** — fact-based progress reports rather than self-celebratory updates.
- **More conversational** — slightly more fluent and colloquial.
- **Less verbose** — may skip detailed summaries for efficiency unless prompted otherwise.

That last one has a practical consequence: Claude may skip a verbal summary after a tool call and go straight to the next action. If you want the visibility back, ask for it: *"After completing a task that involves tool use, provide a quick summary of the work you've done."*

Claude Opus 5 is the documented exception on verbosity. Its default user-facing responses run *longer* than prior models', and — importantly — **raising or lowering `effort` does not reliably change visible response length.** Prompt explicitly for conciseness instead. This is a good example of a rule that belongs to a model rather than to the platform, and Chapter 14's discipline applies: read the per-model page, not the tier.

Two more format defaults worth knowing. Current models **default to LaTeX** for mathematical expressions; if you need plain text you must say so, naming the notations to avoid. And for reports and long-form content the documentation supplies a substantial `<avoid_excessive_markdown_and_bullet_points>` block, whose logic is worth reading even if you never use it verbatim: it tells the model what to write (flowing prose, complete paragraphs), reserves markdown for the cases where it genuinely helps, and gives the reason (*"using prose instead of excessive formatting will improve user satisfaction"*).

20.12 Prompting Is Not Configuration

A prompt cannot do everything, and a recurring failure is trying to make it do a parameter's job. Four controls, four jobs:

Control	Where	What it does
The **prompt**	`system` and `messages`	What the task is, what good looks like, what to produce
`effort`	`output_config`	How much of everything — text, tool calls, thinking — Chapter 18
`thinking`	top level	Whether and how the model reasons before answering — Chapter 18
`max_tokens`	top level	The only hard ceiling. Chapter 19: it reserves no OTPM

Writing "think very carefully and at length about this" into a prompt is not the same as raising `effort`, and it costs you a cache prefix change if you vary it. Conversely, `effort` will not make a response shorter on Opus 5 — that is a prompt's job. The documented guidance points both ways and the boundary is worth learning rather than rediscovering.

There is one genuinely promptable thinking behaviour worth knowing: **adaptive thinking's triggering is steerable.** If the model thinks more often than you want — which the documentation notes can happen with large or complex system prompts — you can say so: *"Thinking adds latency and should only be used when it will meaningfully improve answer quality — typically for problems that require multistep reasoning. When in doubt, respond directly."*

And a small, sharp fact for the exam: **when extended thinking is disabled, Claude Opus 4.5 is particularly sensitive to the word "think" and its variants**; the documentation suggests "consider", "evaluate" or "reason through" instead.

20.13 Model Self-Knowledge

If your application needs Claude to identify itself correctly, or to emit the right model string, say so explicitly. The documented sample prompts:

```
The assistant is Claude, created by Anthropic. The current model is
Claude Opus 5.
```

```
When an LLM is needed, please default to Claude Opus 5 unless the user
requests otherwise. The exact model string for Claude Opus 5 is
claude-opus-5.
```

This is Chapter 14's frozen-weights point arriving in the prompt layer. A model's weights were fixed before some of the models it might be asked about existed, so its knowledge of the current line-up is a training artefact, not a fact it can look up. If the identity matters to your product, put it in the system prompt where it is cheap, cached and correct.

20.14 What Changed, and What to Delete

Prompts written for earlier generations now contain instructions that are inert, counter-productive or fatal. The documented migration list, with the fatal one first:

Prefilling is gone. Starting with Claude 4.6 and Mythos Preview, a prefilled final assistant message returns a **400**. Chapter 16 covered the error; here is what to do about each of the five documented uses:

Prefill was used for	Replacement
Forcing JSON or a classification label	**Structured outputs**, or a tool with an enum field
Eliminating preambles	*"Respond directly without preamble."* Or output inside XML tags. Strip stragglers in post-processing
Steering around bad refusals	Clear prompting in the `user` message; refusals are better calibrated now
Continuing an interrupted response	Move it to the **user** turn: *"Your previous response was interrupted and ended with* `[…]`*. Continue from where you left off."*
Re-injecting context in long chats	Inject into the user turn, or hydrate through tools or compaction

Anti-laziness prompting now over-triggers. If your prompts pushed the model to be thorough or to use tools aggressively, dial that back. *"Where you might have said 'CRITICAL: You MUST use this tool when…', you can use more normal prompting like 'Use this tool when…'."* Blanket defaults become targeted instructions: not *"Default to using [tool]"* but *"Use [tool] when it would enhance your understanding of the problem."* And *"If in doubt, use [tool]"* is specifically called out as a phrase that will cause over-triggering.

Explicit instruction is still needed for action. The models follow instructions precisely, which cuts both ways: *"Can you suggest some changes to improve this function?"* will get suggestions. *"Change this function to improve its performance."* will get changes. If you want a default, the documentation supplies both a `<default_to_action>` block and a `<do_not_act_before_instructions>` block, and which one you want is a product decision, not a prompting one.

Ask for features explicitly. Animations, interactive elements and "go beyond the basics" behaviour are requested, not inferred.

Self-checking has become model-specific. *"Before you finish, verify your answer against [test criteria]"* catches errors reliably on most models — but **Claude Opus 5 verifies its own work well without instruction**, and verification instructions carried over from older prompts cause over-verification, adding tokens and latency. The documented advice when migrating to Opus 5 is to **remove** those instructions rather than rewrite them.

20.15 Visual Analogies

VISUAL ANALOGY

A brilliant new employee

The documentation's own model, and it is a good one. Brilliant: you need not explain what a refund is, or how JSON works, or why a chargeback matters. New: they have never seen your ledger, do not know that "the second system" means the one in Frankfurt, and cannot tell which of your two escalation paths you meant. Everything you would put in a handover note belongs in the prompt; everything you would assume of a competent professional does not.

Where it breaks. A new employee accumulates context: tell them once about the Frankfurt system and they know it in March. The model is stateless (Chapter 16) — every request re-hires them. That is precisely why the handover note lives in a cached prefix rather than in a conversation.

VISUAL ANALOGY

A prompt is a compiled artefact, not a document

You do not hand-write the string you send, any more than you hand-write the bytes of an executable. You declare parts, and something assembles them in an order chosen for the machine rather than for a human reader. The assembled prompt should be *inspectable* — and it will look strange, with the twenty-thousand-token handbook before the two-line task.

Where it breaks. A compiler's output is not meant to be read by anyone; a prompt's is, by the model, and it must still be coherent English in the order it is assembled. You cannot reorder freely — instructions must still follow from what precedes them. The constraint is real: stable first, variable last, and *readable in that order*.

VISUAL ANALOGY

The stranded handbook

Imagine posting a colleague a twenty-page policy manual with every single query you send them, because you insisted on writing the query on page one. They have the manual. You keep paying the postage. Moving the query to the back page costs nothing and the postage drops by two orders of magnitude.

Where it breaks. A real colleague would keep the manual and tell you to stop. The model has no such option: statelessness means it genuinely has nothing between requests, and the "keep it" mechanism is the cache — which only works on the front of the envelope.

20.16 Architecture: The Prompt as a Build Artefact

The failure this architecture prevents is the introduction's: a prompt assembled by string interpolation at the call site, where nobody can see the order and nothing checks it.

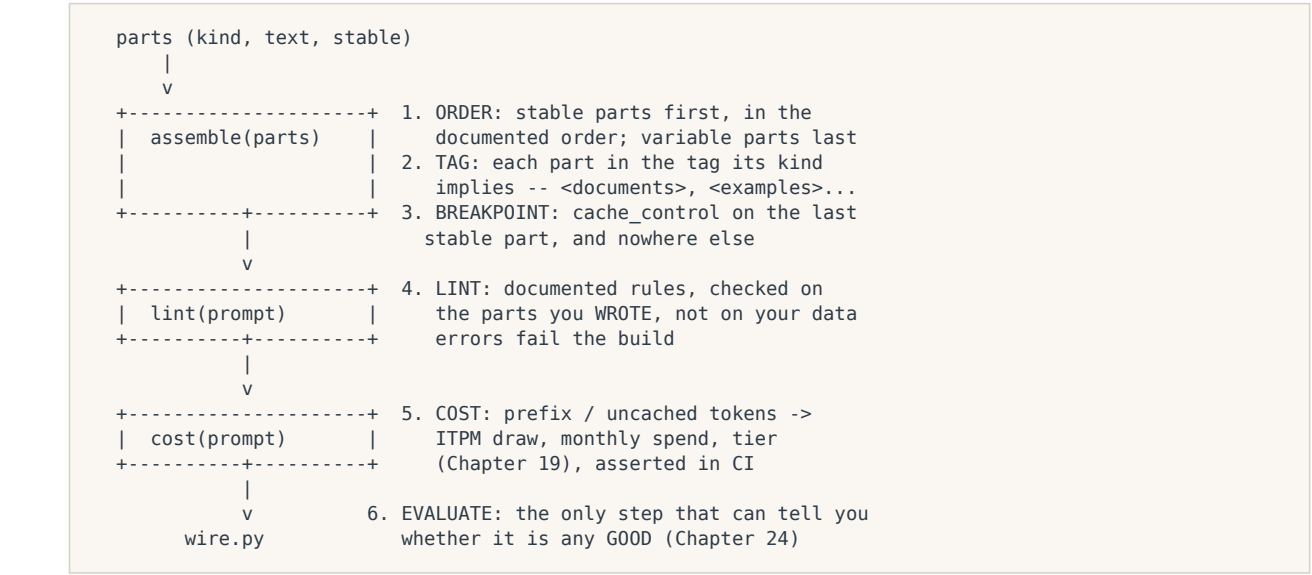


Figure 20.2 - Assembling a prompt instead of writing one

Six responsibilities, and note which one is not in the diagram's remit. Steps 1 to 5 are mechanical, offline, deterministic and testable — this chapter builds all of them. Step 6 needs a model, an evaluation set and a grader, and it is the only step that can tell you whether the prompt *works*. A prompt that lints clean and costs nothing and produces wrong answers is a well-engineered failure. Chapter 24 exists because of that sentence.

20.17 Examples: Four Rewrites

A. Vague to specific.

Before	After
`Summarise this ticket.`	`Summarise this ticket in two sentences of flowing prose. Name the account tier in the first sentence and the requested outcome in the second.`

B. Prohibition to instruction, with a reason.

Before	After
`Do not use bullet points.`	`Write the response as flowing prose paragraphs, because it is rendered into an email body where bullet formatting is lost.`

C. Over-emphasis to normal prompting.

Before	After
`CRITICAL: You MUST ALWAYS use the lookup tool before answering!!!`	`Use the lookup tool when the ticket refers to a specific invoice or account.`

D. Prefill to instruction.

Before	After
A final assistant message of `{"action":`	`Reply with a JSON object and nothing else. Do not begin with a preamble.` — or, better, structured outputs (Chapter 23)

20.18 Code Walkthrough: A Prompt That Knows Its Own Shape

`ch20/parts.py` refuses, from its first line, to treat a prompt as a string.

```
@dataclass(frozen=True)
class Part:
    kind: str
    text: str
    stable: bool          # identical on every request?
```

Three fields. `kind` is one of the six the documentation's recommended tags imply — `instructions`, `output_spec`, `examples`, `documents`, `context`, `query` — and an unknown kind raises rather than being tolerated. `stable` is the field that does the work, and it is the one that no prompt template in the wild records anywhere.

The `Prompt` class then computes four numbers, of which the middle two are the chapter:

```
    @property
    def prefix_tokens(self):
        """Tokens in the longest leading run of stable parts."""
        n = 0
        for p in self.parts:
            if not p.stable:
                break
            n += p.tokens
        return n
```

`break`, not `continue`. That single keyword is prefix-only caching expressed as code: the cacheable region ends at the first variable part and does not resume.

```
    @property
    def stranded_tokens(self):
        seen_variable = False
        n = 0
        for p in self.parts:
            if not p.stable:
                seen_variable = True
                continue
            if seen_variable:
                n += p.tokens
        return n
```

`stranded_tokens` is the number this chapter exists to make visible: stable content that sits *after* the first variable part. It is identical on every request and it can never be cached, so it is billed in full, draws ITPM in full, and does so forever. In the introduction's prompt it is 20,941 tokens.

`uncached_tokens` is simply `variable + stranded`, and it is what `ch19/planner.py` calls the tail. That mapping is the whole bridge between Part 4 and Part 3:

```
def workload_for(prompt, ...):
    return Workload(
        ...,
        prefix_tokens=prompt.prefix_tokens,
        tail_tokens=prompt.uncached_tokens,
        ...)
```

Chapter 19's planner never learns that it is being handed a prompt. It receives a prefix and a tail and does the same arithmetic it did in section 19.26. A prompt's *shape* enters the capacity model as two integers.

`reorder` is deliberately unforgiving:

```
        missing = [p for p in self.parts if p.kind not in kinds]
        if missing:
            raise ValueError(
                f"reorder would drop {len(missing)} part(s): ...")
```

Reordering a prompt must be a permutation. A reorder that silently drops the output specification would look like a large cost saving and would be a bug, so the operation that this chapter's whole argument rests on is the operation that refuses to lose anything.

Finally, `ch20/lint_prompt.py`, and the design decision that makes it usable:

```
AUTHORED = ("instructions", "output_spec", "examples", "context")
def lint_prompt(prompt, long_context_threshold=20_000):
    authored = "\n\n".join(p.text for p in prompt.parts
                           if p.kind in AUTHORED)
    out = list(lint_text(authored))
```

Text rules run over the parts you wrote, never over your documents or the user's input. The policy handbook in the fixture contains the sentence "Do not discuss a chargeback's likely outcome with the customer" — a perfectly good policy, and a `NEGATIVE_WITHOUT_POSITIVE` finding if the linter reads data as if it were instructions. `test_the_linter_reads_your_words_not_your_data` pins this. A linter that produces findings you cannot act on is a linter somebody switches off, and then you have no linter.

20.19 Mini Project: The Prompt Compiler and Linter

What it is. An offline toolkit that treats a prompt as a build artefact: parts with kinds and stabilities, an assembler, a linter carrying documented rules, and a cost model that reuses Chapter 19's planner unchanged. No API key, no network, no recorded model output anywhere.

Folder structure.

```
ch20/
  parts.py          Part, Prompt, prefix / stranded / uncached tokens
  corpus.py         the fixture prompts. Every string is BUILT
  lint_prompt.py    thirteen rules, each citing its guidance
  economics.py      Prompt -> Chapter 19 Workload -> tier and money
  report.py         prints every figure this chapter publishes
  test_prompting.py 33 tests
  wire.py           copied from ch16/ -- est_tokens lives here
  limits.py         copied from ch19/ -- keep in sync
```

```
  planner.py        copied from ch19/ -- keep in sync
```

Running it.

```
cd ch20 && python3 report.py        # under a second
cd ch20 && python3 -m pytest -q     # 33 passed
```

One honest caveat, stated up front. Token counts come from `wire.py`'s `est_tokens`, the four-characters-per-token estimate Chapter 16 introduced. It is an estimate; Chapter 6 explained why the tokenizer is the authority, and Chapter 19 pointed out that `count_tokens` is free and rate-limited independently precisely so you can afford to use it. Every *ratio* in this chapter is robust to the estimate, because both sides are counted the same way. Every absolute figure should be re-measured with `count_tokens` before it goes in a budget.

What this project deliberately does not do. It never claims a prompt is *good*. Sections 20.20 to 20.25 measure shape, order, cost and rule compliance — all of which are computable offline and none of which is quality. A prompt that scores perfectly here can still be wrong, and Chapter 24 is where that gets measured.

20.20 Panel A — Where a Prompt's Tokens Actually Are

```
1. Where a prompt's tokens actually are
---------------------------------------
  documents      20,941 tokens   stable
  instructions      183 tokens   stable
  output_spec        68 tokens   stable
  examples          232 tokens   stable
  query              63 tokens   VARIABLE
  total          21,487 tokens
```

The realistic support-triage prompt from the introduction. Three figures are worth sitting with.

The instructions, the output specification and the examples together are 483 tokens: 2.2 per cent of the prompt. That is the part a team spends its afternoons on, argues about in review and iterates on for weeks. It is a rounding error in the bill.

The ticket — the only thing that changes — is 63 tokens: 0.29 per cent. Three parts in a thousand of every request is new information.

The handbook is 97.5 per cent. Whether it is cached or not is therefore not a detail of the prompt; it *is* the prompt, economically. This is why section 20.22's ordering result is as large as it is: reordering a prompt whose mass is 97.5 per cent stable can only ever be dramatic.

20.21 Panel B — The Same Content, Five Orderings

```
2. The same content, five orderings
-----------------------------------
  arrangement                                          total   prefix  uncached
  A. as written (query before the handbook)         21,487      483    21,004
  B. documented order (handbook first, query last)  21,487   21,424        63
```

```
  C. templated (ticket interpolated into instructions)  21,492        0    21,492
  D. documented order, examples before instructions     21,487   21,424        63
  E. documented order + the rewritten instructions      21,270   21,207        63
```

A, B and D carry byte-identical content — `test_every_arrangement_carries_the_same_content` asserts all three at 21,487 tokens. C differs by five tokens only because interpolating the ticket adds the words "The ticket is:". E is 217 tokens shorter because its instructions were rewritten under section 20.24's rules, which is a nice incidental: the clearer prompt is also the smaller one.

The `prefix` and `uncached` columns are the story. **Arrangement A caches 483 tokens and pays for 21,004 on every request. Arrangement B caches 21,424 and pays for 63.** That is a ratio of **333.4** on the uncached count, produced by moving one part.

Arrangement C is the one worth staring at, because it is the one most likely to be in your codebase. `system = INSTRUCTIONS.format(ticket=ticket)` is tidy, readable, and it makes the first part of the prompt variable — so the cacheable prefix is exactly **zero**, and every one of the 21,492 tokens is uncached on every request, forever.

D exists to make a negative point. Moving the examples above the instructions changes the cost by **nothing at all** — `test_moving_examples_above_instructions_changes_nothing` asserts equality on all three money columns. The ordering that matters is *stable before variable*, not the arrangement of the stable parts among themselves. Chapter 22 has things to say about the arrangement for clarity's sake; it is not where the money is.

20.22 Panel C — What the Ordering Costs

The same five prompts through Chapter 19's planner, on Claude Sonnet 5 at 200,000 requests a day over ten active hours, a peak factor of 2.5, a 90 per cent cache hit rate and a 400-token response.

```
3. What the ordering costs, in Chapter 19's units
-------------------------------------------------
  arrangement                          ITPM/req  bill/req  start ITPM   $/month   tier
  A. as written (query before the han    21,052    21,108      877.2%   203,349 custom
  B. documented order (handbook first     2,205     4,669       91.9%    58,689  scale
  C. templated (ticket interpolated i    21,492    21,492      895.5%   206,730 custom
  D. documented order, examples befor     2,205     4,669       91.9%    58,689  scale
  E. documented order + the rewritten     2,184     4,623       91.0%    58,278  scale
  A -> B is the same content in a different order:
    uncached tokens   21,004 ->    63   (333.4x)
    ITPM per request  21,052 -> 2,205   (9.55x)
    monthly spend    $203,349 -> $58,689   (saving $144,660, 71.1%)
    required tier    custom -> scale
```

Four results.

The ITPM draw falls 9.55×. In Chapter 19's terms, arrangement A runs at **877.2 per cent** of the Start tier's input-tokens-per-minute bucket for Sonnet 5 and cannot be served by any published tier — `tier_by_rate` is `custom`. Arrangement B runs at **91.9 per cent** of the *Start* tier and needs no increase at all. A prompt reordering moved a workload from "phone your account team" to "no action required".

The bill falls 71.1 per cent, $203,349 to $58,689, a saving of **$144,660 a month**. Note that the bill falls by less than the ITPM draw — 4.52× against 9.55× — for exactly the reason Chapter 19 gave: cache reads

cost 0.1× on the invoice and **zero** against the rate limit. Caching is worth more to your throughput than to your bill, and both numbers are large.

Both arrangements still need the Scale tier. `tier_by_rate` for `B` is `start`; `tier_needed` is `scale`, because $58,689 a month is far above Build's $1,000 monthly spend cap. Chapter 19's lesson holds: the rate limits and the spend cap are separate constraints, and improving one does not release the other.

The template is the most expensive thing in the table. Arrangement `C` costs more than the naive arrangement A — $206,730 against $203,349 — because it has no prefix at all, where A at least caches its 483 tokens of instructions. The tidiest-looking code in the chapter is the worst-performing.

20.23 Panel D — The Linter on a Draft and Its Rewrite

A plausible first draft, and the same task rewritten under the documented guidance. Both are BUILT fixtures; neither is a transcript.

```
4. The linter on a first draft and its rewrite
----------------------------------------------
  DRAFT: {'error': 0, 'warn': 4, 'info': 1}
    AMBIGUOUS_OPENING           warn  It should look at the ticket and decide what
    NEGATIVE_WITHOUT_POSITIVE   warn  CRITICAL: You MUST NOT invent policy. NEVER
    OVER_EMPHASIS               warn  CRITICAL
    UNTAGGED_EXAMPLES           warn  like this
    STYLE_MISMATCH              info  3 markdown constructs
  REWRITE: {'error': 0, 'warn': 0, 'info': 0}
```

Every finding maps to a documented sentence.

`AMBIGUOUS_OPENING` — the draft opens *"It should look at the ticket…"*. Who is "it"? This is the golden rule failing on the first line: a colleague with minimal context would ask.

`NEGATIVE_WITHOUT_POSITIVE` — *"You MUST NOT invent policy. NEVER guess a section number."* Both are prohibitions with no positive target and no reason. The rewrite says what to do and why: *"Ground every decision in a quoted policy section, because a triage decision that a human agent cannot check is worse than no decision."* Note that the linter is not against negation as such — `test_a_negative_with_a_reason_is_allowed` confirms that the documentation's own *"never use ellipses, because your response will be read aloud…"* passes clean. The missing reason is the finding.

`OVER_EMPHASIS` — `CRITICAL`, `MUST`, `NEVER`. On the current generation this causes over-triggering, and the documented fix is to write normally. The rule allows technical acronyms through, because `JSON` and `HTTPS` are not shouting; `test_technical_acronyms_are_not_shouting` pins it.

`UNTAGGED_EXAMPLES` — the draft says *"Handle refunds like this"* and then gives one, in prose. The documentation asks for `<example>` tags and three to five of them. `test_three_to_five_examples_is_the_documented_band` checks both ends of the band.

`STYLE_MISMATCH` — the draft asks for *"smoothly flowing prose paragraphs"* and is itself three bullet points. Prompt style influences response style.

The rewrite trips nothing, and it is also 217 tokens shorter than the parts it replaces. That is the usual outcome: prompts get longer when they are unclear, because each new failure adds a sentence rather

than fixing the sentence that caused it.

20.24 Panel E — The Linter on the Five Arrangements

The structural rules, which no amount of good writing can satisfy:

```
5. The linter on the five arrangements
--------------------------------------
  A. as written (query before the handbook)      3  STRANDED_STABLE_CONTENT
                                                    LONG_DATA_AFTER_QUERY
                                                    NO_QUOTE_GROUNDING
  B. documented order (handbook first, query l   1  NO_QUOTE_GROUNDING
  C. templated (ticket interpolated into instr   3  STRANDED_STABLE_CONTENT
                                                    VARIABLE_FIRST
                                                    NO_QUOTE_GROUNDING
  D. documented order, examples before instruc   1  NO_QUOTE_GROUNDING
  E. documented order + the rewritten instruct   0  clean
```

`STRANDED_STABLE_CONTENT` is an **error**, not a warning, and its evidence line is the token count: *"20,941 stable tokens after the first variable part."* It is an error because it is unambiguous, mechanical and expensive, and because there is no case in which you meant to do it.

`VARIABLE_FIRST` is the template bug, caught the moment the part is declared rather than the moment the invoice arrives.

`LONG_DATA_AFTER_QUERY` carries the quality argument rather than the money one — *queries at the end improved response quality by up to 30 per cent in tests* — and it fires on exactly the same arrangement. Two independent rules, one fix, which is the chapter's thesis in two lines of linter output.

`NO_QUOTE_GROUNDING` fires on any prompt carrying 20,000+ document tokens that never asks for quotes. It is informational, and it is the only finding on the otherwise-correct arrangement B. `test_quote_grounding_is_suggested_only_above_the_threshold` confirms it stays quiet on small documents, because a rule that fires on everything is noise.

20.25 Panel F — The One Arrangement That Is Both Clean and Cheapest

```
6. The one arrangement that is clean and cheapest
-------------------------------------------------
  E. documented order + the rewritten instructions
    findings 0 | prefix 21,207 | ITPM/req 2,184 | $58,278/month
    411 dollars a month cheaper than B on shorter instructions alone
```

`test_only_one_arrangement_is_completely_clean` asserts that E is the only one with an empty finding list, and `test_the_clean_arrangement_is_also_the_cheapest` asserts that it is also the minimum-cost arrangement of the five.

That coincidence is the point of the chapter, and it is not really a coincidence. Clarity removes words. Removing words removes tokens. Correct ordering caches the tokens that remain. The three principles pull in the same direction, and $411 a month — the difference between B and E — is what 217 tokens of

unnecessary instruction costs at this volume. It is small next to the $144,660 that ordering is worth, and it is not nothing, and it arrived as a side effect of writing more clearly.

20.26 Evaluation and Deployment

In CI. The linter's errors should fail a build. `STRANDED_STABLE_CONTENT`, `VARIABLE_FIRST`, `PLACEHOLDER_IN_STABLE_PART` and `LEGACY_THINKING` are all mechanical, all unambiguous and all expensive; none of them is a matter of taste. The warnings should be reported and not fail, because a team that cannot merge until a linter is happy will find a way to make it happy that does not involve improving the prompt.

Assert the cost, not just the shape. The most valuable single test in `test_prompting.py` is arguably `test_ordering_changes_the_itpm_draw_by_over_nine_times`, because it is the one that would fail if somebody "tidied up" the assembly order in six months. A prompt's cacheable prefix is a production characteristic; put a number on it in a test and it stops being able to drift silently.

Re-measure with `count_tokens`. Before any figure from this toolkit reaches a budget or a rate limit increase request, replace `est_tokens` with the real endpoint. It is free, it is exact, and Chapter 19 established that its rate limit is independent of the Messages API — 2,000 to 8,000 RPM depending on tier — precisely so that you can afford to call it for everything.

Then evaluate. Everything above is necessary and none of it is sufficient. Chapter 24 builds the regression suite that answers the only question that matters, which is whether the prompt produces correct output on cases you care about. Ship the linter and the cost assertion in week one because they are cheap; do not mistake them for evidence.

20.27 Hands-on Labs

Lab 1 — Find your stranded tokens. Take a real prompt from your own system. Split it into parts, mark each one stable or variable, and compute `prefix_tokens` and `stranded_tokens`. **Success:** you can state, in tokens, how much stable content sits after your first variable part.

Lab 2 — Price the reordering. Feed both orderings of that prompt through `economics.costed` with your real traffic. **Success:** two monthly figures and a required tier for each.

Lab 3 — Add a rule. Write a linter rule for a documented principle not yet covered — for example, a prompt asking Claude to "think" while thinking is disabled on Opus 4.5. **Success:** the rule fires on a fixture that violates it and stays silent on three that do not.

Lab 4 — Break the linter deliberately. Change `lint_prompt` to lint the whole rendered prompt rather than only the authored parts. Re-run Panel E. **Success:** you can name the false positives that appear and explain which part of the fixture caused each.

Lab 5 — Rewrite the draft yourself. Without looking at `REWRITE`, rewrite `DRAFT` until the linter is clean. Then compare token counts. **Success:** zero findings, and a statement of whether your version is longer or shorter than the draft, and why.

20.28 Lab Solutions

Lab 1. The common answer in real systems is "all of it": conversation history, retrieved documents and tool definitions are typically appended after the user's current message, which makes them stranded by construction. Chapter 30 covers where the cache breakpoints belong once tools are in the picture; the immediate win is usually moving retrieved documents above the query.

Lab 2. Use `workload_for(prompt, requests_per_day=..., output_tokens=...)` and read `tier_by_rate` and `monthly`. If the two orderings give the same tier, your document is small relative to your query and this chapter's effect is genuinely minor for you — which is a legitimate finding and worth recording rather than assuming.

Lab 3. The rule needs the model as an input, which is the point: `lint_text` cannot express it, because it is a *per-model* fact and belongs with `ch14/registry.py`. The clean implementation adds `lint_for_model(prompt, model_id)` and reads the registry, raising `Unverified` for any model whose sensitivity is not recorded — the discipline Chapter 14 established.

Lab 4. Three false positives appear. `NEGATIVE_WITHOUT_POSITIVE` fires on the handbook's "Do not discuss a chargeback's likely outcome"; `OVER_EMPHASIS` fires once the handbook's capitalised tier names accumulate; `STYLE_MISMATCH` fires because the ticket text is prose while the instructions are numbered. None is actionable, all three would be filed as linter bugs by the first engineer who saw them, and the linter would be disabled within a sprint.

Lab 5. Almost everyone's rewrite is shorter than the draft. Unclear prompts grow by accretion: each observed failure adds a clause, and the clause that caused the failure is never removed. `REWRITE` is 217 tokens shorter than the three parts it replaces and trips no rules.

20.29 Best Practices

BEST PRACTICE

Order stable before variable, always

It is the documented quality ordering and the caching ordering at the same time, and on a document-heavy prompt it is worth an order of magnitude on the ITPM bucket. There is no case where the reverse is better.

BEST PRACTICE

Never interpolate user input into a system prompt

`INSTRUCTIONS.format(ticket=ticket)` sets your cacheable prefix to zero and, from Chapter 58's perspective, mixes untrusted input into the highest-authority region of the request. Two unrelated reasons, one rule.

BEST PRACTICE

Give the reason with the rule

A bare prohibition covers the case it names. A prohibition with a reason generalises to the cases you did not think of. This is the highest ratio of benefit to effort in the whole chapter.

BEST PRACTICE

Say what to do

"Write flowing prose paragraphs" beats "do not use markdown", and the documentation's first formatting recommendation is exactly this.

BEST PRACTICE

Three to five examples, tagged, relevant and diverse

And check that they are diverse: if every example resolves the same way, you have taught that resolution rather than the decision.

BEST PRACTICE

Delete anti-laziness prompting when you migrate

"CRITICAL: You MUST" becomes "Use this tool when". "If in doubt, use [tool]" is specifically documented as causing over-triggering. Migration is a subtraction exercise more often than an addition one.

BEST PRACTICE

Ask Claude to critique your prompt's examples

The documentation suggests it, it costs one request, and it catches the diversity problem that humans reliably miss in their own examples.

20.30 Common Mistakes

COMMON MISTAKE

Putting the query above the documents

Symptom. A prompt that reads well, a cache hit rate near zero, and an ITPM bucket that binds at a fraction of your request rate. **Cause.** Caching is prefix-only. Stable content after the first variable part is stranded. **Fix.** Documents first, query last — which is also the documented quality ordering. **Detection.** `stranded_tokens > 0`. Section 20.22 prices it at $144,660 a month.

COMMON MISTAKE

Interpolating variables into the system prompt

Symptom. Zero cache hits despite a large stable prompt; cost identical to having no cache at all. **Cause.** A variable first part means a cacheable prefix of zero. **Fix.** Keep the system prompt literal. Put the variable in the last user message. **Detection.** `VARIABLE_FIRST`, an error, raised before the request is built.

COMMON MISTAKE

Prohibitions without alternatives

Symptom. The model avoids the named behaviour and substitutes something equally unwanted. **Cause.** A prohibition defines a boundary, not a target. **Fix.** Name the target and give the reason, so the instruction generalises. **Detection.** `NEGATIVE_WITHOUT_POSITIVE`, which passes any negation carrying a reason.

COMMON MISTAKE

Shouting at a model that already listens

Symptom. Tools that used to under-trigger now fire constantly; responses over-thorough. **Cause.** Anti-laziness prompting written for earlier generations. Current models over-trigger on "CRITICAL", "You MUST", "If in doubt, use [tool]". **Fix.** Normal prose. "Use this tool when…". Lower `effort` as a fallback. **Detection.** `OVER_EMPHASIS`. Review every prompt at every model migration.

COMMON MISTAKE

Carrying prefills forward

Symptom. A 400 on the first request after upgrading to a 4.6-or-later model. **Cause.** Prefilled final assistant messages are no longer supported. **Fix.** Structured outputs for format, direct instruction for preambles, a user-turn continuation for interruptions. **Detection.** Chapter 16's preflight, or the linter's `LEGACY_THINKING` sibling rules.

COMMON MISTAKE

Prompting for what a parameter does

Symptom. "Think very carefully and at length" in a prompt, alongside an `effort` setting nobody has touched. **Cause.** Confusing the prompt with the configuration. Chapter 18's four controls have four jobs. **Fix.** Task and quality bar in the prompt; depth in `effort`; ceiling in `max_tokens`. **Detection.** Grep your prompts for "think", "carefully", "thorough" and check what `effort` is set to.

COMMON MISTAKE

Carrying self-verification instructions to Opus 5

Symptom. Longer, slower, more expensive responses after migrating, with no accuracy gain. **Cause.** Opus 5 verifies its own work well without instruction; inherited verification prompts cause over-verification. **Fix.** **Remove** them rather than rewriting them, per the documented migration advice. **Detection.** Compare output token counts before and after migration on the same evaluation set.

COMMON MISTAKE

Believing a clean linter means a good prompt

Symptom. A prompt that passes every mechanical check and produces wrong answers. **Cause.** Shape is not quality. Nothing in this chapter measures correctness. **Fix.** Chapter 24's evaluation suite. The linter is a floor, not a ceiling. **Detection.** An evaluation set. There is no substitute and there is no shortcut.

20.31 Performance Considerations

Order is the largest single lever in this chapter, and it costs nothing: 9.55× on the ITPM draw and 71.1 per cent of the bill on the fixture workload.

Clarity is a latency lever. Shorter, unambiguous instructions produce shorter responses, and Chapter 11's rule stands: output tokens dominate latency, cost five times input, and are metered as produced. Asking explicitly for two sentences is faster than asking for "a summary" and getting six.

Examples are prefix cost, not per-request cost — provided they are in the stable prefix. Three to five well-chosen examples that live in a cached prefix are close to free after the first request. The same examples pasted below the user's question cost full price every time.

Quote-grounding trades output tokens for reliability. Asking for quotes first adds output tokens and adds latency. On a long-document task it is usually worth it, and it is a real cost that should be measured rather than assumed away.

Prompt changes invalidate the cache. Chapter 18 established this for the thinking configuration and Chapter 11 for the prefix generally: editing a system prompt starts a new prefix, so the first request after every deployment pays the write multiplier of 1.25×. Deploy prompt changes deliberately, not continuously.

20.32 Security Considerations

Never interpolate untrusted input into the system prompt. The performance argument is section 20.21's; the security argument is stronger. Chapter 7 established the principal hierarchy — Anthropic, operator, user — and Chapter 16 that the system prompt is where operator authority lives. Pasting a user's text into it grants that text operator authority. Chapter 58 develops prompt injection properly; the rule to adopt now is that the system prompt is a **literal**, and user content arrives in a user message.

Structure is a defence. Wrapping untrusted content in `<document>` or `<input>` tags and telling the model that content inside them is data rather than instruction is not a complete defence, and it is a meaningful one. Chapter 22 covers the technique; Chapter 58 covers its limits honestly.

Examples leak. A few-shot example built from a real customer record puts that record in every request, in a cached prefix, for as long as the prompt lives. Synthesise examples, or scrub them properly.

A prompt is a credential-adjacent artefact. It encodes your policies, your escalation thresholds and often your internal system names. It belongs in version control with review, not in a database row that anybody with production access can edit.

20.33 Real Production Examples

A. The tidy refactor. A team moved their retrieved documents from above the user's question to below it, so the prompt "read in the order a person would ask it". Cache hit rate fell from 94 per cent to 3 per cent overnight. The pull request was two lines and had two approvals.

B. The helpful template. A prompt library rendered every system prompt through a formatter for consistency. One template took a `{{today}}` date variable in its first line. The prefix changed once a day,

which sounds harmless; it meant every request before the first cache write of the day paid full price, and the team's traffic was concentrated in the first hour.

C. The escalating instructions. Eighteen months of accreted guidance, each clause added after an incident, none removed. By the end the system prompt contained four separate instructions about tool use, two of which contradicted each other and one of which said "If in doubt, use the search tool". After migration to a current model the search tool fired on nearly every request. The fix removed 40 per cent of the prompt.

D. The example that taught the wrong lesson. Three few-shot examples for a classifier, all drawn from the same week's tickets, all resolving to `escalate` because that week had a billing incident. The model escalated everything. Diversity is not a style note.

20.34 Certification Notes

Concept	Exam domain	Why it is examinable
The golden rule	D4 Prompt Engineering & Structured Output	The documented test for prompt clarity.
Be clear and direct; ask for "above and beyond"	D4	Vague prompts get vague outputs by design.
Motivation generalises; bare rules do not	D4	The documentation's ellipses example.
Say what to do, not what not to do	D4	The first formatting recommendation.
Examples: relevant, diverse, structured, 3–5	D4	A count the exam can ask for directly.
Longform data at the top, query at the bottom	D5 Context Management & Reliability	Up to 30% quality, and the cache prefix.
Prefix-only caching and stranded content	D5	Where prompting meets Chapter 19.
Quote grounding for long documents	D5	The documented long-context technique.
Roles in the system prompt	D4	Even one sentence changes behaviour.
Prompt versus `effort` versus `max_tokens`	D1 Agentic Architecture & Orchestration	Four controls, four jobs.
Prefill removal and its five replacements	D4	A migration fact with a 400 attached.
Anti-laziness prompting now over-triggers	D1	Migration is subtraction.
Opus 5 verbosity and self-verification exceptions	D4	Per-model, not per-tier.

Concept	Exam domain	Why it is examinable
Model self-knowledge prompts	D4	Frozen weights meet the current line-up.

20.35 Twenty Practice Questions

1. State the golden rule of clear prompting.
2. What mental model does the documentation offer for how to treat Claude?
3. Give the documented rewrite of "Create an analytics dashboard".
4. Why is "NEVER use ellipses" weaker than the version with a reason?
5. What should you write instead of "Do not use markdown in your response"?
6. Name the three properties good examples should have.
7. How many examples does the documentation recommend?
8. Where should longform data go in a prompt, and by how much can it matter?
9. What is a "stranded" token, and why is it the most expensive kind?
10. What does interpolating a variable into the first part of a prompt do to caching?
11. What is the documented technique for grounding a long-document task?
12. Where does the system prompt go in a Messages API request?
13. Which current model's visible response length does `effort` not reliably change?
14. What do current models default to for mathematical expressions?
15. Name three documented replacements for prefilling.
16. What phrase is specifically called out as causing tool over-triggering?
17. Which model is documented as particularly sensitive to the word "think" when thinking is disabled?
18. When migrating to Opus 5, what should you do with inherited self-verification instructions?
19. Name a success criterion that prompt engineering is *not* the best tool for.
20. What must exist before you begin prompt engineering at all?

20.36 Answers and Explanations

1. **Show your prompt to a colleague with minimal context and ask them to follow it. If they'd be confused, Claude will be too.**
2. **A brilliant but new employee who lacks context on your norms and workflows.** Brilliant covers the domain; new covers your systems.
3. Add the modifiers: *"Include as many relevant features and interactions as possible. Go beyond the basics to create a fully-featured implementation."* Above-and-beyond behaviour must be requested.
4. Because a rule with a reason **generalises**. Told that the output is read aloud, the model also avoids tables and emoji — things you did not prohibit.

5. *"Your response should be composed of smoothly flowing prose paragraphs."* A positive target, not a boundary.
6. **Relevant, diverse and structured** — mirroring the use case, covering edge cases, wrapped in `<example>` tags.
7. **Three to five.**
8. **At the top, above the query, instructions and examples.** Queries at the end improved response quality **by up to 30 per cent** in tests on complex multi-document inputs.
9. A **stable token that sits after the first variable part**. It is identical on every request and uncacheable, because prefix caching already ended, so it is billed and rate-limited in full forever.
10. It makes the cacheable prefix **zero**. Section 20.21 measures it as the most expensive of five arrangements.
11. Ask Claude to **quote the relevant passages first, in `<quotes>` tags**, before performing the task.
12. It is a **top-level `system` field**, not a message — with the documented mid-conversation `"role": "system"` exception on Fable 5, Mythos 5, Opus 4.8 and Opus 5.
13. **Claude Opus 5.** Its responses run longer than prior models' and effort does not reliably change visible length; prompt for conciseness explicitly.
14. **LaTeX.** Ask for plain text explicitly, naming the notations to avoid.
15. Any three of: **structured outputs** for format; a **tool with an enum** for classification; a **direct instruction** against preambles; a **user-turn continuation** for interrupted responses; **tool-based or compaction-based context hydration**.
16. **"If in doubt, use [tool]."** Also the "CRITICAL: You MUST" family generally.
17. **Claude Opus 4.5.** Use "consider", "evaluate" or "reason through".
18. **Remove them**, not rewrite them. Opus 5 verifies its own work well, and inherited instructions cause over-verification.
19. **Latency and cost**, which the documentation notes can sometimes be improved more easily by selecting a different model. Chapter 10's triage adds: missing information, exact execution, external state and checkability.
20. **Success criteria, a way to test against them empirically, and a first draft prompt.**

20.37 Ten Practical Exercises

1. Add a `context` part to the fixture prompt and confirm it changes no cost figure when placed before the query and changes several when placed after it.
2. Write `explain(prompt)` that prints one sentence naming the most expensive structural problem and its price per month.
3. Extend `lint_text` with a rule for the documented LaTeX default: a prompt asking for plain-text maths without naming the notations to avoid.
4. Add a `diversity(examples)` check that flags a set of examples whose outputs are all identical.
5. Implement `suggest_order(parts)` returning the cheapest valid ordering, and prove it equals the documented ordering on the fixture.

6. Add a rule for prefilled assistant messages, using `ch16/wire.py`'s `NO_PREFILL` set.
7. Compute the break-even document size at which reordering saves less than one dollar a month at your traffic.
8. Add per-model rules by reading `ch14/registry.py`, raising `Unverified` for any model whose behaviour is not recorded.
9. Replace `est_tokens` with a `count_tokens` client stub and quantify the estimate's error on each fixture part.
10. Write a `git diff`-style prompt differ that reports which changes would invalidate the cache prefix.

20.38 Five Challenge Labs

1. **The prompt compiler.** Build a real assembler that takes parts, emits the request body with `cache_control` on the last stable part, and refuses to build if any structural rule is violated.
2. **The accretion detector.** Given a prompt's git history, identify clauses added after an incident and never revisited, and measure their token cost over time.
3. **The migration linter.** Given a source model and a target model, list every clause in a prompt that the documentation says to change, remove or add.
4. **The example auditor.** Score a few-shot set for relevance and diversity mechanically, then compare your score to Claude's own critique of the same set.
5. **The evaluation bridge.** Wire this chapter's linter to Chapter 24's regression suite so a prompt change reports shape, cost and accuracy in one output.

20.39 Ten Interview Questions

1. A colleague moves the retrieved documents below the user's question because it reads better. What do you say, and what number do you bring?
2. Why does giving a reason with an instruction outperform the instruction alone?
3. When is prompt engineering the wrong tool?
4. What does `INSTRUCTIONS.format(ticket=ticket)` cost, and why?
5. Where does a role belong, and why there?
6. How would you decide between a prompt change and a tool?
7. What would you delete from a prompt written for Claude 3?
8. How do you tell whether a long-document failure is retrieval or reasoning?
9. Your linter is clean and the output is wrong. What is your next step?
10. Design the CI checks for a repository whose main artefact is a prompt.

20.40 Summary

Prompt engineering is usually presented as a list of tricks, and it is better understood as a handful of principles that each have a measurable consequence. The documentation's own framing is the right starting point: treat Claude as a brilliant but new employee, and apply the golden rule — show the

prompt to a colleague with minimal context, and if they would be confused, so will the model be. Under that sit four principles. **Be specific**, including about behaviour you might expect to be inferred; "above and beyond" has to be asked for. **Give the reason with the rule**, because a bare prohibition covers only the case it names while an explained one generalises to the cases you did not think of. **Say what to do rather than what not to do**, which is the documentation's first recommendation for steering format. And **show examples** — three to five, relevant, diverse and wrapped in `<example>` tags — because they are the most reliable lever on format, tone and structure that exists.

The reframing this chapter adds is that a prompt is not a string but an ordered list of parts, each with a kind and, crucially, a **stability**. That second property is almost never recorded anywhere in real systems, and it decides the economics of the whole request. Prompt caching is prefix-only, so the cacheable region ends at the first part that changes, and every stable token after that point is **stranded**: identical on every request, uncacheable, billed and rate-limited in full, forever. On a realistic support-triage prompt this is not a rounding error. The instructions, output specification and examples together are 483 tokens — 2.2 per cent of the prompt. The ticket is 63 tokens, 0.29 per cent. The policy handbook is 97.5 per cent, and whether it sits above or below the ticket is therefore the single largest decision in the prompt.

Section 20.22 puts the number on it. Identical content, 21,487 tokens either way: with the handbook last, 21,004 uncached tokens per request, 877.2 per cent of the Start tier's ITPM bucket, a required tier of Custom and **$203,349 a month**. With the handbook first, 63 uncached tokens, 91.9 per cent of Start, and **$58,689** — a **9.55×** reduction in ITPM draw and a saving of **$144,660 a month, 71.1 per cent**, from a change no reviewer would call substantive. The template that interpolates the ticket into the instructions is worse still at $206,730, because a variable first part means a cacheable prefix of exactly zero. And the ordering that produces all of this is the ordering Anthropic already documents for an entirely different reason: longform data at the top, query at the bottom, worth **up to 30 per cent** in response quality on complex multi-document inputs. The quality rule and the money rule are the same rule.

Two boundaries close the chapter. The first is between prompting and configuration. The prompt says what the task is and what good looks like; `effort` says how much of everything; `thinking` says whether to reason first; `max_tokens` is the only hard ceiling. Writing "think carefully at length" is not a substitute for effort, and on Claude Opus 5 effort is documented not to reliably change visible response length — for that, you prompt. Migration is largely a subtraction exercise: prefills are gone and return a 400, anti-laziness language now over-triggers, and inherited self-verification instructions should be **removed** rather than rewritten when moving to Opus 5.

The second boundary is the honest one. Everything this chapter measures — shape, order, cost, rule compliance — is computable offline, deterministic and testable, and **none of it is quality**. The linter's thirteen rules can tell you that a prompt breaks documented guidance; they cannot tell you that it works. Arrangement E is clean and cheapest and might still produce wrong answers on a quarter of your tickets, and nothing in this chapter would notice. That is not a weakness of the toolkit; it is the reason Chapter 24 exists, and the reason the documentation puts success criteria and empirical tests *before* prompt engineering rather than after it. Get the shape right because it is cheap and mechanical. Then go and measure whether it is any good.

KEY TAKEAWAYS

1. **The golden rule:** if a colleague with minimal context would be confused, so will Claude. It is the only principle here a machine cannot check.
2. **Ask for what you want**, including "above and beyond" behaviour — it is not inferred from a vague prompt.
3. **Give the reason with the rule.** An explained instruction generalises; a bare prohibition covers only the case it names.
4. **Say what to do, not what not to do**, and match your prompt's style to the response style you want.
5. **Three to five examples**, relevant, diverse and wrapped in `<example>` tags. Diversity is not a style note — uniform examples teach the uniform answer.
6. **A prompt is parts with kinds and stabilities**, not a string. The stability is the field nobody records and the one that decides the bill.
7. **Stable before variable.** Stranded stable content is the most expensive content there is: $144,660 a month on the fixture workload, for a reordering.
8. **Never interpolate user input into the system prompt.** Cacheable prefix zero, and operator authority granted to untrusted text.
9. **The documented quality ordering and the caching ordering are the same ordering** — long data at the top, query at the bottom.
10. **A clean linter is a floor, not evidence.** Nothing in this chapter measures whether the prompt is any good; Chapter 24 does.

20.41 Glossary

Term	Definition
Prompt engineering	Improving results by changing what you send, as distinct from which model or which parameters.
Golden rule	If a colleague with minimal context would be confused by your prompt, Claude will be too.
Few-shot / multishot prompting	Steering output by including worked examples, ideally three to five, in `<example>` tags.
Part	One span of a prompt, with a kind and a stability.
Stable part	Content byte-identical on every request; the only kind that can live in a cache prefix.
Variable part	Content that changes per request; it ends the cacheable prefix.
Cacheable prefix	The longest leading run of stable parts. Caching is prefix-only.
Stranded tokens	Stable tokens placed after the first variable part: uncacheable, and paid for on every request.
Long context prompting	Guidance for inputs of ~20,000 tokens and up: data at the top, query at the bottom.
Quote grounding	Asking Claude to quote relevant passages in `<quotes>` tags before performing the task.

Term	Definition
Role prompting	A sentence in the system prompt setting Claude's role; even one sentence changes behaviour.
Over-emphasis	"CRITICAL: You MUST" language that causes over-triggering on current models.
Anti-laziness prompting	Instructions written to push earlier models to be thorough; now a source of over-triggering.
Prefill	A partial final assistant message. Unsupported from Claude 4.6; returns 400.
Model self-knowledge	Telling Claude which model it is, and which model string to emit.
Prompt linter	A mechanical check of documented rules. Checks shape, never quality.

20.42 Further Reading

Primary sources, at `platform.claude.com/docs/en/build-with-claude/prompt-engineering/`, verified **1 August 2026**:

- `claude-prompting-best-practices` — the living reference: general principles, output and formatting, tool use, thinking, agentic systems and migration.
- `overview` — when prompt engineering is the right solution, and what must exist before you start.
- `prompting-claude-opus-5`, `prompting-claude-sonnet-5`, `prompting-claude-fable-5`, `prompting-claude-opus-4-8` — the per-model behavioural differences, including Opus 5's verbosity and over-verification exceptions.
- `/docs/en/test-and-evaluate/develop-tests` — defining success criteria and building evaluations, the prerequisite this chapter assumes.
- The interactive prompting tutorial on GitHub, and the metaprompt notebook in the Claude Cookbook for generating a first draft.

Where to go next in this book. Chapter 21 takes the system prompt seriously: roles, operator authority and what belongs above the conversation. **Chapter 22** develops XML structuring, the technique this chapter has used without justifying. **Chapter 23** covers structured output and JSON contracts, which is where "ask for JSON" becomes "guarantee JSON". **Chapter 24** builds the evaluation and regression suite that turns every claim in this chapter from an opinion into a measurement. **Chapter 25** covers guardrails, validation and prompt debugging. **Chapter 30** returns to prompt caching properly, including where the breakpoints go once tools and conversation history are in the request. **Chapter 52** treats the context budget as a first-class constraint, where this chapter's 97.5-per-cent document becomes a design problem rather than a caching one. **Chapter 58** covers prompt injection, and is the reason section 20.32 insists the system prompt is a literal.